Universidade Estácio de Sá

Desenvolvimento Full Stack

Sistema cadastral com Java, JPA, EJB, Servlets e JSP

DGT2811 · Desenvolvimento Back-End Corporativo com Java e Cloud

Yan Martins Lourenço Fraga

Matrícula: 202211246051

Campus: Cabo Frio - Braga

Turma: não se aplica · Semestre informado: 4/5

https://github.com/Marti-yan/Estacio-DGT2811

Cabo Frio · 23 de outubro de 2026

1. Objetivo da prática

Implementar um cadastro Web de produtos com persistência no SQL Server, regras de negócio em EJB e apresentação em JSP. O trabalho reúne os três procedimentos do roteiro: persistência e controle, interface cadastral e melhoria visual com Bootstrap.

O sistema permite consultar, incluir, alterar e excluir produtos. Cada produto possui identificador, nome, quantidade e preço de venda. A implementação usa orientação a objetos e mantém as responsabilidades separadas sem criar camadas que apenas repassariam chamadas.

2. Ambiente e decisões de implementação

O ambiente validado utilizou JDK 17, Ant 1.10.15, GlassFish 7.1.1, Jakarta EE 10, SQL Server 2022 e o driver Microsoft JDBC 12.10.2.jre11. As telas utilizam Bootstrap 5.3.8 por CDN. O SQL Server foi executado localmente em contêiner e o GlassFish, no Windows.

O roteiro menciona Jakarta EE 8 e, ao mesmo tempo, pede a troca de imports javax por jakarta. Essa combinação é inconsistente. Foi usada uma versão posterior do GlassFish, permitida no próprio enunciado, com a API Jakarta EE 10. O descritor de persistência usa o esquema XML 3.0, também utilizado pela especificação Jakarta Persistence 3.1. O tipo javax.sql.DataSource permanece assim porque pertence ao Java SE.

O banco da prática anterior não foi disponibilizado. Por isso, o projeto inclui um script com a tabela Produto, única tabela necessária aos fluxos solicitados. Não foram inventadas entidades de outros exercícios. O campo precoVenda é Float no Java e REAL no SQL Server, conforme o tipo exigido pelo roteiro. Para aplicações financeiras, BigDecimal e DECIMAL seriam mais adequados devido à precisão decimal.

O projeto usa Ant e mantém os módulos CadastroEE, CadastroEE-ejb e CadastroEE-war. A configuração do NetBeans é um projeto Ant de formato livre, com as duas áreas de fontes separadas. Essa é uma adaptação ao assistente de criação do roteiro: preserva a organização e os artefatos EAR, EJB e WAR, mas evita arquivos gerados desnecessários. A compilação e a implantação foram executadas por linha de comando; não se afirma uma execução do assistente gráfico do NetBeans.

A porta 8080 já estava ocupada no computador de validação. O GlassFish foi configurado na porta 18080 e o SQL Server na porta 14330. O caminho da aplicação permaneceu CadastroEE-war. O guia de execução explica a configuração padrão e essa variação.

3. Procedimento 1 - Persistência e controle

O arquivo database/loja.sql cria a base loja, a tabela dbo.Produto e três produtos de exemplo quando a tabela está vazia. O identificador é gerado pelo SQL Server. O script não apaga objetos já existentes. O usuário loja recebeu apenas SELECT, INSERT, UPDATE e DELETE sobre a tabela.

A classe Produto representa os dados e valida nome, quantidade e preço. A atualização valida todos os valores antes de modificar o objeto, evitando alterações parciais quando um campo é inválido. As anotações JPA mapeiam os atributos para a tabela existente, sem geração automática de esquema.

ProdutoFacade é um EJB sem estado, exposto pela interface local ProdutoFacadeLocal. O contêiner injeta o EntityManager e gerencia as transações JTA. A inclusão usa persist, a exclusão usa remove e a alteração modifica uma entidade gerenciada. O JPA sincroniza essa alteração ao concluir a transação; uma chamada adicional a merge não é necessária nesse caso.

O recurso JNDI jdbc/loja aponta para SQLServerPool. O script de configuração gera um descritor temporário a partir da senha informada na sessão, registra os recursos e testa a conexão. A senha não faz parte do código nem do repositório.

ServletProduto recebe o EJB com @EJB, consulta os produtos e encaminha a lista para ProdutoTeste.jsp. O resultado é uma lista HTML com nome, quantidade e preço. A montagem da apresentação fica no JSP para não misturar HTML ao código Java do servlet.

4. Procedimento 2 - Interface cadastral

ServletProdutoFC implementa o Front Controller. Seu método processRequest recebe o parâmetro acao e trata listar, formIncluir, formAlterar, incluir, alterar e excluir. O servlet interpreta a requisição, chama a interface local e define os atributos que serão lidos pela página.

ProdutoLista.jsp apresenta identificador, nome, quantidade e preço em uma tabela, além dos links de inclusão, alteração e exclusão. ProdutoDados.jsp atende tanto a inclusão quanto a alteração, com campos ocultos para acao e, quando necessário, id. Os dados são enviados por POST.

O destino dos formulários é ProdutoDados.jsp e o destino das listagens é ProdutoLista.jsp. A navegação usa RequestDispatcher.forward, como solicitado. As páginas ficam dentro de WEB-INF para que o acesso passe pelo servlet.

O link Excluir abre uma confirmação; somente o POST confirmado remove o produto. Os formulários usam um token de sessão contra requisições forjadas. Após uma gravação o token é renovado para impedir repetição ao atualizar a página. Como consequência, um formulário aberto anteriormente em outra aba também precisa ser reaberto.

O sistema rejeita nome vazio ou longo demais, quantidades negativas ou não inteiras e preços negativos, não numéricos, infinitos ou NaN. IDs inválidos retornam 400 e produtos inexistentes retornam 404. Os valores exibidos pelo usuário passam por c:out, evitando interpretação de HTML. Dados de conexão e detalhes de exceções não são exibidos na página de erro geral.

5. Procedimento 3 - Bootstrap

O cabeçalho compartilhado inclui o CSS e o JavaScript do Bootstrap por CDN, com verificação de integridade, além da metatag viewport. As duas páginas cadastrais incluem esse cabeçalho, evitando repetir os mesmos links.

A listagem usa container, table, table-striped e table-dark. Os links usam btn-primary ou btn-danger e os tamanhos pedidos. O formulário usa form, mb-3, form-label e form-control. A tabela fica em table-responsive para permitir rolagem horizontal em telas estreitas. Rótulos associados aos campos, cabeçalhos de tabela e mensagens com role ajudam na acessibilidade.

A interface usa somente recursos necessários ao cadastro. Não foram adicionados frameworks de front-end, APIs paralelas ou classes genéricas sem uso.

6. Resultados da execução

A compilação gerou CadastroEE-ejb.jar, CadastroEE-war.war e CadastroEE.ear. A implantação do EAR no GlassFish e o ping de SQLServerPool foram concluídos com sucesso. A base permaneceu com os três exemplos após a limpeza dos produtos de teste.

Foram aprovadas 14 verificações do domínio e 29 verificações HTTP. Os testes HTTP passaram pela aplicação implantada, usando EJB, JPA e SQL Server reais. Eles verificaram listagem, inclusão, recuperação, alteração, exclusão, validações, acentuação UTF-8, escape de HTML, ações desconhecidas, IDs inválidos e proteção dos formulários. O script cria apenas seus próprios dados de teste e os remove ao terminar.

Na demonstração visual foi incluído um Estojo com 12 unidades e preço de R$ 24,90. Em seguida, a quantidade foi alterada para 15 e o preço para R$ 26,90. Por fim, o produto foi excluído após a confirmação. As capturas mostram esses resultados e a consulta pelo servlet inicial. Os identificadores não são reiniciados após a exclusão, comportamento normal de uma coluna IDENTITY.

As evidências estão na pasta docs/evidencias. As figuras e os registros de execução são apresentados a seguir no PDF. Não foram usados resultados simulados nem imagens de interfaces que não correspondam à aplicação.

Evidências da execução



Figura 1. Listagem inicial com os três produtos da base, preços formatados e ações do cadastro.

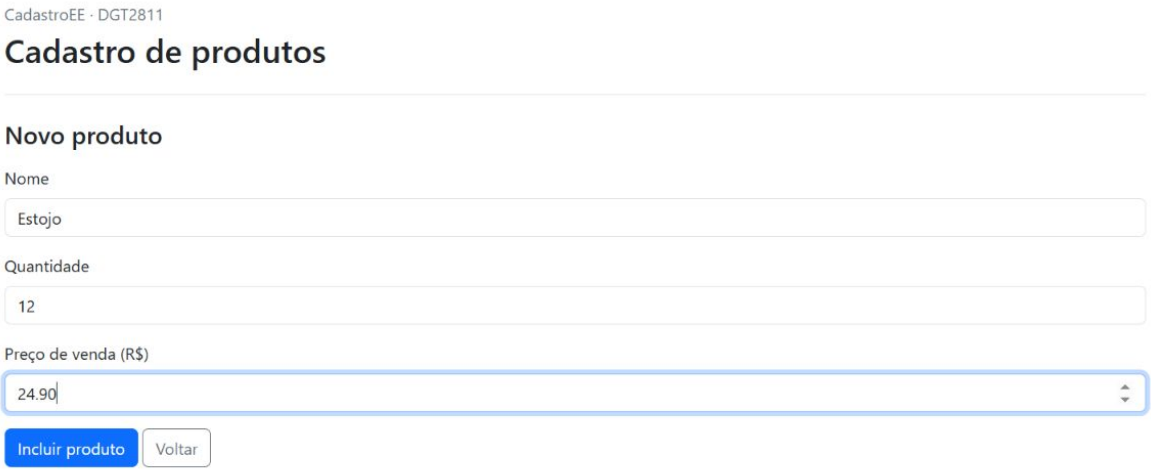


Figura 2. Formulário preenchido para inclusão do Estojo, com 12 unidades e preço de R$ 24,90.

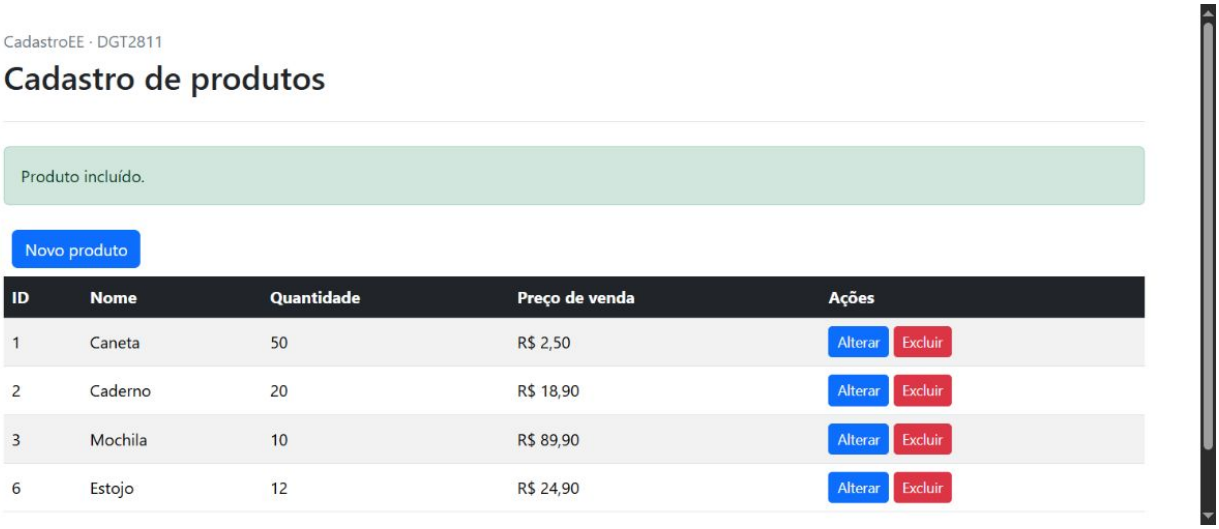


Figura 3. Inclusão concluída: o Estojo aparece na listagem, com confirmação da operação.

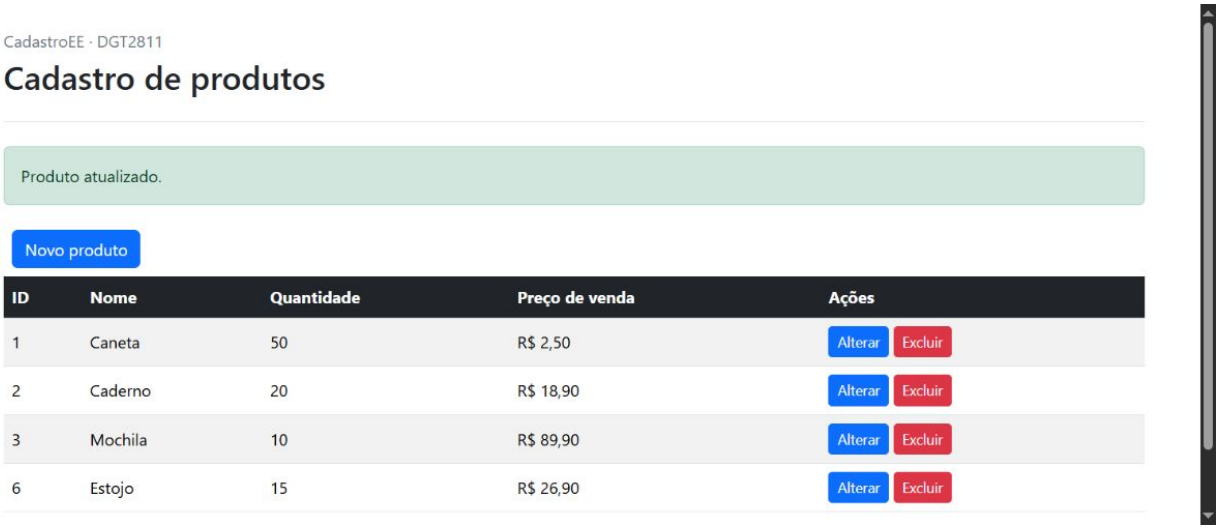


Figura 4. Alteração concluída: o Estojo passa a ter 15 unidades e preço de R$ 26,90.

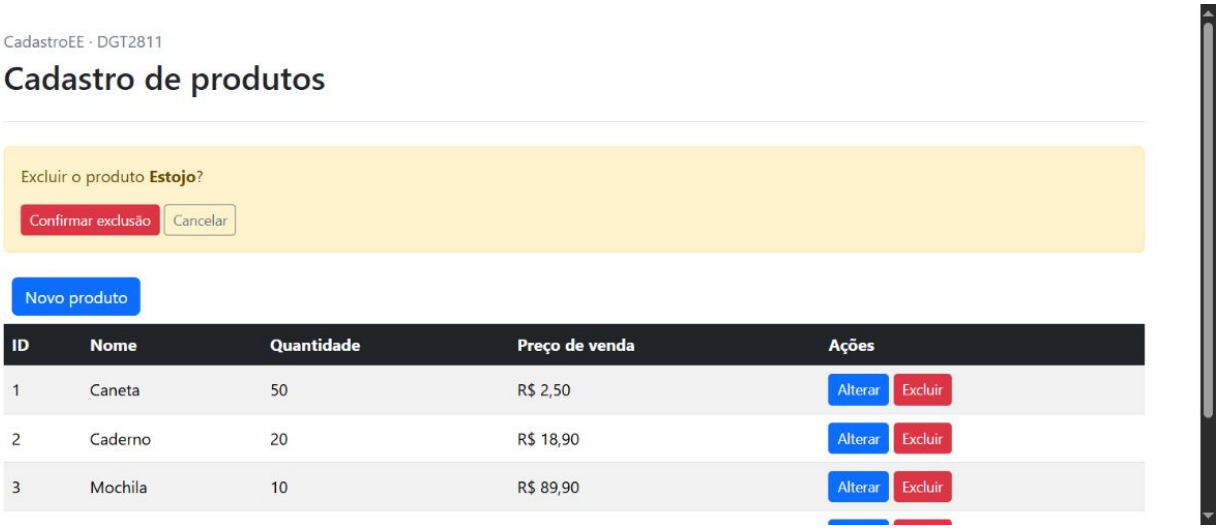


Figura 5. O link Excluir abre a confirmação. O produto só é removido após o envio por POST.

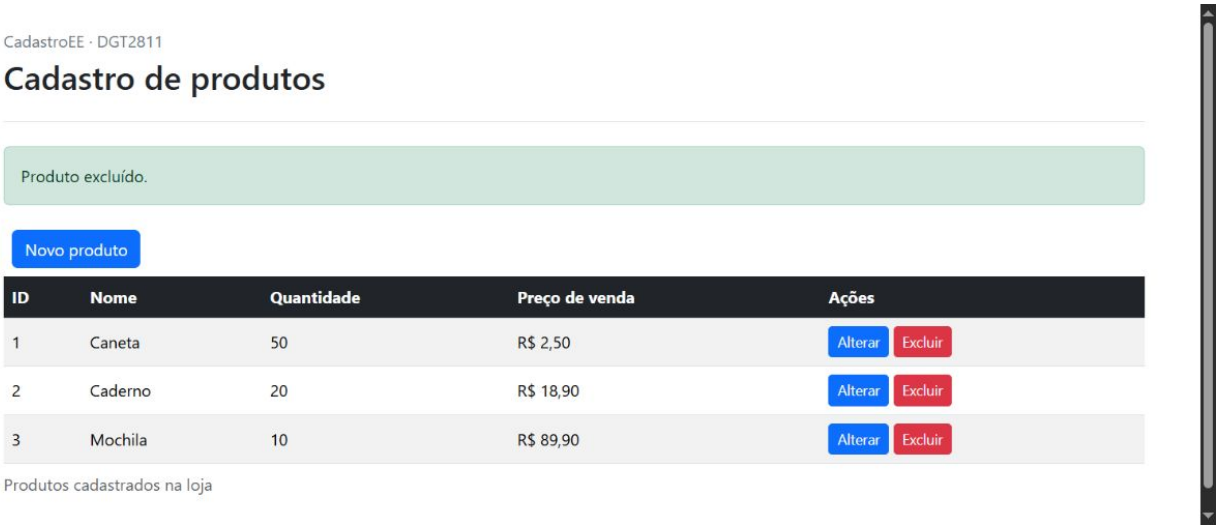


Figura 6. Exclusão concluída. A listagem volta a apresentar os três produtos iniciais.

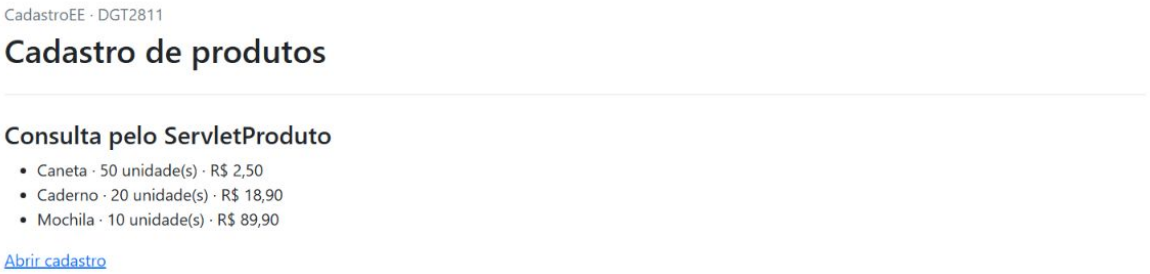


Figura 7. ServletProduto consulta o EJB e apresenta os produtos em uma lista HTML.

Registros da validação

Servidor, pool e aplicação

```
Version = Eclipse GlassFish 7.1.1 (commit: 8e05816bdc1d25b10d4274e0beea417d4ad412cc)
Command version executed successfully.
Command ping-connection-pool executed successfully.
CadastroEE  <ear, ejb, web>
Command list-applications executed successfully.
```

Consulta ao banco após os testes

```
idProduto   nome
                   quantidade  precoVenda
----------- ---------------------------------------------------------------------------------------
----------------- ----------- --------------
          1 Caneta
                              50            2.5
          2 Caderno
                              20           18.9
          3 Mochila
                              10      89.900002

(3 rows affected)
```

Verificações HTTP

```
OK - Listagem com JSP e EJB
OK - Bootstrap na listagem
OK - Acentuação UTF-8 na listagem
OK - Formulário de inclusão
OK - Acentuação UTF-8 no formulário
OK - Inclusão persistida
OK - Nome escapado contra XSS
OK - Reenvio do mesmo formulário bloqueado
OK - Formulário recupera a entidade
OK - Alteração persistida e preço com vírgula
OK - Nova consulta confirma os valores gravados
OK - GET de exclusão apenas pede confirmação
OK - Validação: nome=
OK - Validação: nome=xxxxxxxxxxxxxxxxxxxx
OK - Validação: quantidade=-1
OK - Validação: quantidade=1.5
OK - Validação: quantidade=2147483648
OK - Validação: precoVenda=-1
OK - Validação: precoVenda=NaN
OK - Validação: precoVenda=Infinity
OK - POST sem token rejeitado
OK - Inclusão por GET rejeitada
OK - Alteração por GET rejeitada
OK - Ação desconhecida rejeitada
OK - ID inválido rejeitado
OK - Produto inexistente retorna 404
OK - Servlet de teste funciona
OK - Exclusão persistida
OK - Consulta confirma a exclusão

29 verificações HTTP passaram.
```

O build Ant também executou 14 verificações do domínio com sucesso e gerou o EAR implantado.

7. Análise e conclusão - Procedimento 1

Como é organizado um projeto corporativo no NetBeans?

Uma aplicação corporativa pode reunir módulos EJB e Web em um EAR. O módulo EJB contém entidades e serviços de negócio; o módulo Web contém servlets, JSPs e recursos de apresentação. O projeto principal descreve como esses módulos são empacotados. Neste trabalho, o build Ant realiza essa composição e a configuração da IDE aponta para as fontes de cada módulo.

Qual é o papel de JPA e EJB em uma aplicação Web Java?

JPA faz o mapeamento entre objetos e dados relacionais, oferecendo operações de persistência e consultas. EJB administra componentes de negócio no servidor, incluindo transações, ciclo de vida e concorrência das instâncias. Aqui, ProdutoFacade usa JPA dentro de transações controladas pelo GlassFish; o servlet não precisa abrir conexões nem controlar commit e rollback.

Como o NetBeans melhora a produtividade com JPA e EJB?

Os assistentes podem gerar entidades a partir de tabelas, interfaces e session beans, além de ajudar com descritores, bibliotecas, conclusão de código e implantação. Esse código inicial precisa ser revisado: nomes, tipos, regras de validação e versões das APIs devem corresponder à aplicação. O trabalho mantém uma implementação pequena, organizada para edição na IDE e também compilável fora dela.

O que são Servlets e qual é o suporte oferecido pela IDE?

Servlets são componentes executados pelo contêiner Web para receber requisições e produzir respostas. HttpServlet fornece métodos como doGet e doPost. O NetBeans ajuda a criar classes, mapear URLs, editar web.xml e executar o projeto no servidor. Neste cadastro, os mapeamentos estão explicitamente no descritor Web.

Como os Servlets se comunicam com os Session Beans?

A anotação @EJB solicita a injeção de uma referência à interface local. O servlet chama seus métodos e o contêiner encaminha a chamada a uma instância gerenciada do bean. O código não deve instanciar ProdutoFacade com new, pois isso dispensaria o gerenciamento de transações e a injeção do EntityManager.

8. Análise e conclusão - Procedimento 2

Como funciona o Front Controller na arquitetura MVC?

Um controlador central recebe as requisições de um conjunto de funcionalidades, escolhe a operação e encaminha os dados para a apresentação. No projeto, ServletProdutoFC é o controlador, Produto e o serviço EJB representam o modelo, e os JSPs são as visões. Essa divisão concentra a navegação e evita regras de negócio espalhadas pelas páginas.

Quais são as diferenças e semelhanças entre Servlets e JSPs?

Ambos são executados no servidor e participam do processamento de requisições Web. Um servlet é escrito principalmente em Java e é adequado ao controle do fluxo. Um JSP é escrito principalmente como apresentação, combinando HTML, Expression Language e bibliotecas de tags. O contêiner traduz e compila o JSP em um servlet; portanto, eles compartilham a infraestrutura Web, mas têm papéis diferentes no código-fonte.

Qual é a diferença entre redirecionamento e forward? Para que servem parâmetros e atributos?

Um redirecionamento envia ao navegador uma resposta que provoca uma nova requisição e normalmente altera a URL exibida. Os atributos da requisição anterior não acompanham essa nova chamada. O forward transfere o processamento internamente, preservando a mesma requisição e seus atributos, sem mudar a URL no navegador. Parâmetros chegam como texto da URL ou do formulário; atributos são objetos colocados pelo servidor, como a lista de produtos e a entidade a editar.

9. Análise e conclusão - Procedimento 3

Como o Bootstrap é utilizado?

Os arquivos CSS e JavaScript são incluídos na página, e as classes do framework são aplicadas aos elementos HTML. Neste trabalho, a maior parte do resultado vem das classes CSS de tabelas, formulários, espaçamento, alertas e botões. O JavaScript foi incluído conforme o roteiro, embora as operações do cadastro sejam processadas no servidor.

Por que o Bootstrap favorece a independência estrutural do HTML?

O HTML descreve conteúdo e semântica; as classes CSS concentram regras de apresentação em folhas externas. Isso permite ajustar cores, espaços e comportamento visual sem reescrever a lógica de negócio. Essa independência não é absoluta: alguns componentes exigem marcação e classes específicas. O benefício principal é separar melhor apresentação, conteúdo e processamento.

Qual é a relação entre Bootstrap e responsividade?

O framework oferece estilos mobile first, pontos de quebra, contêineres e utilitários que se adaptam à largura disponível. A metatag viewport permite o dimensionamento correto em dispositivos móveis. No cadastro, os campos ocupam a largura do contêiner e a tabela usa table-responsive; quando o espaço é insuficiente, a rolagem fica na tabela em vez de forçar toda a página a uma largura fixa.

10. Conclusão geral

O cadastro demonstrou a integração entre a camada Web, os serviços EJB e a persistência JPA no SQL Server. A separação de responsabilidades manteve o código pequeno e permitiu verificar regras do produto sem servidor, complementando essa verificação com testes HTTP da aplicação completa.

Os três procedimentos foram reunidos em uma aplicação executável e documentada. As adaptações de versão, de criação do projeto Ant e de esquema inicial do banco foram explicitadas. O ambiente é destinado à prática local; uma publicação de uso real exigiria autenticação, autorização, certificado adequado e tratamento monetário com precisão decimal.

11. Referências

Roteiro da disciplina DGT2811: https://sway.cloud.microsoft/xgFrBfFns0eB9W2U?ref=Link

GlassFish 7.1.1 e Jakarta EE 10: https://glassfish.org/download_gf7.html

Jakarta Persistence 3.1 e seus esquemas XML: https://jakarta.ee/specifications/persistence/3.1/

Bootstrap 5.3 - introdução e CDN: https://getbootstrap.com/docs/5.3/getting-started/introduction/

Microsoft JDBC Driver para SQL Server: https://learn.microsoft.com/pt-br/sql/connect/jdbc/download-microsoft-jdbc-driver-for-sql-server

Logotipo institucional extraído da capa do Relatório de Autoavaliação Institucional 2024 da Estácio: https://cdn.portal.estacio.br/Relatorio_de_Autoavaliacao_Institucional_de_2024_e_2023_998242a87e.pdf

Consulta às fontes e execução: 23/10/2026.

12. Código-fonte

O apêndice do PDF contém os códigos da aplicação, os descritores, o script do banco, a configuração do ambiente e os testes. Os arquivos originais, com a estrutura de diretórios e o histórico de versões, estão no repositório indicado na capa.

CadastroEE-ejb/src/java/cadastroee/controller/ProdutoFacade.java

```
 1  package cadastroee.controller;
 2
 3  import cadastroee.model.Produto;
 4  import jakarta.ejb.Stateless;
 5  import jakarta.persistence.EntityManager;
 6  import jakarta.persistence.PersistenceContext;
 7  import java.util.List;
 8
 9  @Stateless
10  public class ProdutoFacade implements ProdutoFacadeLocal {
11
12      @PersistenceContext(unitName = "CadastroEE-ejbPU")
13      private EntityManager em;
14
15      @Override
16      public void create(Produto produto) {
17          em.persist(produto);
18      }
19
20      @Override
21      public void edit(int id, String nome, int quantidade, float precoVenda) {
22          find(id).atualizar(nome, quantidade, precoVenda);
23      }
24
25      @Override
26      public void remove(int id) {
27          em.remove(find(id));
28      }
29
30      @Override
31      public Produto find(int id) {
32          Produto produto = em.find(Produto.class, id);
33          if (produto == null) {
34              throw new ProdutoNaoEncontradoException();
35          }
36          return produto;
37      }
38
39      @Override
40      public List<Produto> findAll() {
41          return em.createQuery("SELECT p FROM Produto p ORDER BY p.idProduto", Produto.class)
42                  .getResultList();
43      }
44  }
```

CadastroEE-ejb/src/java/cadastroee/controller/ProdutoFacadeLocal.java

```
 1  package cadastroee.controller;
 2
 3  import cadastroee.model.Produto;
 4  import jakarta.ejb.Local;
 5  import java.util.List;
 6
 7  @Local
 8  public interface ProdutoFacadeLocal {
 9      void create(Produto produto);
10      void edit(int id, String nome, int quantidade, float precoVenda);
11      void remove(int id);
12      Produto find(int id);
13      List<Produto> findAll();
14  }
```

CadastroEE-ejb/src/java/cadastroee/controller/ProdutoNaoEncontradoException.java

```
 1  package cadastroee.controller;
 2
 3  import jakarta.ejb.ApplicationException;
 4
 5  @ApplicationException(rollback = true)
 6  public class ProdutoNaoEncontradoException extends RuntimeException {
 7      public ProdutoNaoEncontradoException() {
 8          super("Produto não encontrado. Ele pode ter sido excluído em outra aba.");
 9      }
10  }
```

CadastroEE-ejb/src/java/cadastroee/model/Produto.java

```
 1  package cadastroee.model;
 2
 3  import jakarta.persistence.Column;
 4  import jakarta.persistence.Entity;
 5  import jakarta.persistence.GeneratedValue;
 6  import jakarta.persistence.GenerationType;
 7  import jakarta.persistence.Id;
 8  import jakarta.persistence.Table;
 9  import java.io.Serializable;
10
11  @Entity
12  @Table(name = "Produto", schema = "dbo")
13  public class Produto implements Serializable {
14
15      @Id
16      @GeneratedValue(strategy = GenerationType.IDENTITY)
17      @Column(name = "idProduto")
18      private Integer idProduto;
19
20      @Column(nullable = false, length = 100)
21      private String nome;
22
23      @Column(nullable = false)
24      private Integer quantidade;
25
26      @Column(nullable = false)
27      private Float precoVenda;
28
29      protected Produto() {
30          // Exigido pelo JPA.
31      }
32
33      public Produto(String nome, Integer quantidade, Float precoVenda) {
34          atualizar(nome, quantidade, precoVenda);
35      }
36
37      public void atualizar(String nome, Integer quantidade, Float precoVenda) {
38          if (nome == null || nome.isBlank() || nome.strip().length() > 100) {
39              throw new IllegalArgumentException("Informe um nome de até 100 caracteres.");
40          }
41          if (quantidade == null || quantidade < 0) {
42              throw new IllegalArgumentException("A quantidade deve ser um inteiro maior ou
    igual a zero.");
43          }
44          if (precoVenda == null || !Float.isFinite(precoVenda) || precoVenda < 0) {
45              throw new IllegalArgumentException("O preço deve ser um número maior ou igual a
    zero.");
46          }
47          this.nome = nome.strip();
48          this.quantidade = quantidade;
49          this.precoVenda = precoVenda;
50      }
51
52      public Integer getIdProduto() {
53          return idProduto;
54      }
55
```

```
56      public String getNome() {
57          return nome;
58      }
59
60      public Integer getQuantidade() {
61          return quantidade;
62      }
63
64      public Float getPrecoVenda() {
65          return precoVenda;
66      }
67  }
```

CadastroEE-ejb/src/conf/persistence.xml

```
 1  <?xml version="1.0" encoding="UTF-8"?>
 2  <persistence xmlns="https://jakarta.ee/xml/ns/persistence" version="3.0"
 3               xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
 4               xsi:schemaLocation="https://jakarta.ee/xml/ns/persistence
    https://jakarta.ee/xml/ns/persistence/persistence_3_0.xsd">
 5      <persistence-unit name="CadastroEE-ejbPU" transaction-type="JTA">
 6          <jta-data-source>jdbc/loja</jta-data-source>
 7          <class>cadastroee.model.Produto</class>
 8          <exclude-unlisted-classes>true</exclude-unlisted-classes>
 9          <properties>
10              <property name="jakarta.persistence.schema-generation.database.action"
    value="none"/>
11              <property name="eclipselink.target-database" value="SQLServer"/>
12              <property name="eclipselink.cache.shared.default" value="false"/>
13          </properties>
14      </persistence-unit>
15  </persistence>
```

CadastroEE-war/src/java/cadastroee/servlets/ServletProduto.java

```
 1  package cadastroee.servlets;
 2
 3  import cadastroee.controller.ProdutoFacadeLocal;
 4  import jakarta.ejb.EJB;
 5  import jakarta.servlet.ServletException;
 6  import jakarta.servlet.http.HttpServlet;
 7  import jakarta.servlet.http.HttpServletRequest;
 8  import jakarta.servlet.http.HttpServletResponse;
 9  import java.io.IOException;
10
11  public class ServletProduto extends HttpServlet {
12
13      @EJB
14      private ProdutoFacadeLocal facade;
15
16      @Override
17      protected void doGet(HttpServletRequest request, HttpServletResponse response)
18              throws ServletException, IOException {
19          request.setAttribute("produtos", facade.findAll());
20          request.getRequestDispatcher("/WEB-INF/views/ProdutoTeste.jsp").forward(request,
    response);
21      }
22  }
```

CadastroEE-war/src/java/cadastroee/servlets/ServletProdutoFC.java

```
 1  package cadastroee.servlets;
 2
 3  import cadastroee.controller.ProdutoFacadeLocal;
 4  import cadastroee.controller.ProdutoNaoEncontradoException;
 5  import cadastroee.model.Produto;
 6  import jakarta.ejb.EJB;
 7  import jakarta.servlet.ServletException;
 8  import jakarta.servlet.http.HttpServlet;
 9  import jakarta.servlet.http.HttpServletRequest;
10  import jakarta.servlet.http.HttpServletResponse;
11  import java.io.IOException;
```

```
12  import java.util.Set;
13  import java.util.UUID;
14
15  public class ServletProdutoFC extends HttpServlet {
16
17      private static final Set<String> ACOES = Set.of(
18              "listar", "formIncluir", "formAlterar", "incluir", "alterar", "excluir");
19
20      @EJB
21      private ProdutoFacadeLocal facade;
22
23      @Override
24      protected void doGet(HttpServletRequest request, HttpServletResponse response)
25              throws ServletException, IOException {
26          processRequest(request, response);
27      }
28
29      @Override
30      protected void doPost(HttpServletRequest request, HttpServletResponse response)
31              throws ServletException, IOException {
32          processRequest(request, response);
33      }
34
35      protected void processRequest(HttpServletRequest request, HttpServletResponse response)
36              throws ServletException, IOException {
37          request.setCharacterEncoding("UTF-8");
38          response.setContentType("text/html;charset=UTF-8");
39          response.setHeader("Cache-Control", "no-store");
40          String acao = request.getParameter("acao");
41          if (acao == null) {
42              acao = "listar";
43          }
44          if (!ACOES.contains(acao)) {
45              response.sendError(400, "Ação inválida.");
46              return;
47          }
48          boolean post = "POST".equals(request.getMethod());
49          if (!post && ("incluir".equals(acao) || "alterar".equals(acao))) {
50              response.setHeader("Allow", "POST");
51              response.sendError(405);
52              return;
53          }
54          var session = request.getSession();
55          if (session.getAttribute("csrf") == null) {
56              session.setAttribute("csrf", UUID.randomUUID().toString());
57          }
58          if (post && !session.getAttribute("csrf").equals(request.getParameter("csrf"))) {
59              response.sendError(403, "Formulário expirado. Abra a listagem e tente
    novamente.");
60              return;
61          }
62
63          String destino = "ProdutoLista.jsp";
64          try {
65              switch (acao) {
66                  case "formIncluir" -> destino = "ProdutoDados.jsp";
67                  case "formAlterar" -> {
68                      request.setAttribute("produto", facade.find(id(request)));
69                      destino = "ProdutoDados.jsp";
70                  }
71                  case "incluir", "alterar" -> {
72                      Produto dados = dados(request);
73                      if ("incluir".equals(acao)) {
74                          facade.create(dados);
75                      } else {
76                          facade.edit(id(request), dados.getNome(), dados.getQuantidade(),
    dados.getPrecoVenda());
77                      }
78                      request.setAttribute("mensagem", "incluir".equals(acao)
79                              ? "Produto incluído." : "Produto atualizado.");
80                      session.setAttribute("csrf", UUID.randomUUID().toString());
81                  }
```

```
                case "excluir" -> {
                    if (post) {
                        facade.remove(id(request));
                        request.setAttribute("mensagem", "Produto excluído.");
                        session.setAttribute("csrf", UUID.randomUUID().toString());
                    } else {
                        request.setAttribute("exclusao", facade.find(id(request)));
                    }
                }
                default -> { }
            }
        } catch (ProdutoNaoEncontradoException e) {
            response.setStatus(404);
            request.setAttribute("erro", e.getMessage());
        } catch (IllegalArgumentException e) {
            response.setStatus(400);
            request.setAttribute("erro", e.getMessage());
            if ("incluir".equals(acao) || "alterar".equals(acao)) {
                request.setAttribute("reexibir", true);
                request.setAttribute("editando", "alterar".equals(acao));
                destino = "ProdutoDados.jsp";
            }
        }
        if ("ProdutoLista.jsp".equals(destino)) {
            request.setAttribute("produtos", facade.findAll());
        }
        request.getRequestDispatcher("/WEB-INF/views/" + destino).forward(request,
    response);
    }

    private int id(HttpServletRequest request) {
        try {
            int id = Integer.parseInt(request.getParameter("id"));
            if (id > 0) {
                return id;
            }
        } catch (NumberFormatException ignored) {
            // A mesma mensagem atende valores ausentes ou inválidos.
        }
        throw new IllegalArgumentException("Identificador de produto inválido.");
    }

    private Produto dados(HttpServletRequest request) {
        try {
            int quantidade = Integer.parseInt(request.getParameter("quantidade"));
            String preco = request.getParameter("precoVenda");
            float precoVenda = Float.parseFloat(preco == null ? "" : preco.replace(',',
    '.'));
            return new Produto(request.getParameter("nome"), quantidade, precoVenda);
        } catch (NumberFormatException e) {
            throw new IllegalArgumentException("Informe uma quantidade inteira e um preço
    numérico válidos.");
        }
    }
}
```

CadastroEE-war/web/WEB-INF/views/Erro.jsp

```
<%@ page pageEncoding="UTF-8" %>
<%@ include file="cabecalho.jspf" %>
<div class="alert alert-danger" role="alert">Não foi possível concluir a operação. Tente
novamente em instantes.</div>
<a href="${controller}?acao=listar">Voltar à listagem</a>
<%@ include file="rodape.jspf" %>
```

CadastroEE-war/web/WEB-INF/views/ProdutoDados.jsp

```
<%@ page pageEncoding="UTF-8" %>
<%@ include file="cabecalho.jspf" %>
<c:set var="alterando" value="${not empty produto or editando}"/>
<c:set var="acao" value="${alterando ? 'alterar' : 'incluir'}"/>
<h2 class="h4 mb-3">${alterando ? 'Alterar produto' : 'Novo produto'}</h2>
```

```
<form class="form" method="post" action="${controller}">
    <input type="hidden" name="acao" value="${acao}">
    <input type="hidden" name="csrf" value="${sessionScope.csrf}">
    <c:if test="${alterando}">
        <input type="hidden" name="id" value="<c:out value='${reexibir ? param.id : produto.idProduto}'/>">
    </c:if>
    <div class="mb-3">
        <label class="form-label" for="nome">Nome</label>
        <input class="form-control" id="nome" name="nome" required maxlength="100"
               value="<c:out value='${reexibir ? param.nome : produto.nome}'/>">
    </div>
    <div class="mb-3">
        <label class="form-label" for="quantidade">Quantidade</label>
        <input class="form-control" id="quantidade" name="quantidade" type="number" min="0" max="2147483647" step="1" required
               value="<c:out value='${reexibir ? param.quantidade : produto.quantidade}'/>">
    </div>
    <div class="mb-3">
        <label class="form-label" for="precoVenda">Preço de venda (R$)</label>
        <input class="form-control" id="precoVenda" name="precoVenda" type="number" min="0" step="0.01" required
               value="<c:out value='${reexibir ? param.precoVenda : produto.precoVenda}'/>">
    </div>
    <button class="btn btn-primary" type="submit">${alterando ? 'Salvar alterações' : 'Incluir produto'}</button>
    <a class="btn btn-outline-secondary" href="${controller}?acao=listar">Voltar</a>
</form>
<%@ include file="rodape.jspf" %>
```

CadastroEE-war/web/WEB-INF/views/ProdutoLista.jsp

```
<%@ page pageEncoding="UTF-8" %>
<%@ include file="cabecalho.jspf" %>
<c:if test="${not empty exclusao}">
    <section class="alert alert-warning" aria-label="Confirmar exclusão">
        <p>Excluir o produto <strong><c:out value="${exclusao.nome}"/></strong>?</p>
        <form method="post" action="${controller}">
            <input type="hidden" name="acao" value="excluir">
            <input type="hidden" name="id" value="${exclusao.idProduto}">
            <input type="hidden" name="csrf" value="${sessionScope.csrf}">
            <button class="btn btn-danger btn-sm" type="submit">Confirmar exclusão</button>
            <a class="btn btn-outline-secondary btn-sm" href="${controller}?acao=listar">Cancelar</a>
        </form>
    </section>
</c:if>
<a class="btn btn-primary m-2" href="${controller}?acao=formIncluir">Novo produto</a>
<div class="table-responsive">
    <table class="table table-striped align-middle">
        <caption>Produtos cadastrados na loja</caption>
        <thead class="table-dark">
            <tr><th scope="col">ID</th><th scope="col">Nome</th><th scope="col">Quantidade</th>
                <th scope="col">Preço de venda</th><th scope="col">Ações</th></tr>
        </thead>
        <tbody>
        <c:forEach var="produto" items="${produtos}">
            <tr>
                <td>${produto.idProduto}</td>
                <td><c:out value="${produto.nome}"/></td>
                <td>${produto.quantidade}</td>
                <td class="text-nowrap"><fmt:formatNumber value="${produto.precoVenda}" type="currency" currencySymbol="R$"/></td>
                <td class="text-nowrap">
                    <a class="btn btn-primary btn-sm" href="${controller}?acao=formAlterar&amp;id=${produto.idProduto}">Alterar</a>
                    <a class="btn btn-danger btn-sm" href="${controller}?acao=excluir&amp;id=${produto.idProduto}">Excluir</a>
                </td>
            </tr>
        </c:forEach>
```

```
        <c:if test="${empty produtos}">
            <tr><td colspan="5" class="text-center text-secondary py-4">Nenhum produto cadastrado.</td></tr>
        </c:if>
        </tbody>
    </table>
</div>
<%@ include file="rodape.jspf" %>
```

CadastroEE-war/web/WEB-INF/views/ProdutoTeste.jsp

```
<%@ page pageEncoding="UTF-8" %>
<%@ include file="cabecalho.jspf" %>
<h2 class="h4">Consulta pelo ServletProduto</h2>
<ul>
    <c:forEach var="produto" items="${produtos}">
        <li><c:out value="${produto.nome}"/> · ${produto.quantidade} unidade(s) · 
            <fmt:formatNumber value="${produto.precoVenda}" type="currency" currencySymbol="R$"/></li>
    </c:forEach>
</ul>
<a href="${controller}?acao=listar">Abrir cadastro</a>
<%@ include file="rodape.jspf" %>
```

CadastroEE-war/web/WEB-INF/views/cabecalho.jspf

```
<%@ page contentType="text/html; charset=UTF-8" pageEncoding="UTF-8" %>
<%@ taglib prefix="c" uri="jakarta.tags.core" %>
<%@ taglib prefix="fmt" uri="jakarta.tags.fmt" %>
<!doctype html>
<html lang="pt-BR">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Cadastro de produtos</title>
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/bootstrap.min.css" rel="stylesheet"
    integrity="sha384-sRIl4kxILFvY47J16cr9ZwB07vP4J8+LH7qKQnuqkuIAvNWLzeN8tE5YBujZqJLB" crossorigin="anonymous">
    <script defer src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/js/bootstrap.bundle.min.js"
    integrity="sha384-FKyoEForCGlyvwx9Hj09JcYn3nv7wiPVlz7YYwJrWVcXK/BmnVDxM+D2scQbITxI" crossorigin="anonymous"></script>
</head>
<body class="container py-4">
<fmt:setLocale value="pt_BR"/>
<c:url var="controller" value="/ServletProdutoFC"/>
<header class="border-bottom mb-4 pb-3">
    <p class="text-secondary mb-1">CadastroEE · DGT2811</p>
    <h1 class="h2">Cadastro de produtos</h1>
</header>
<main>
<c:if test="${not empty erro}">
    <div class="alert alert-danger" role="alert"><c:out value="${erro}"/></div>
</c:if>
<c:if test="${not empty mensagem}">
    <div class="alert alert-success" role="status"><c:out value="${mensagem}"/></div>
</c:if>
```

CadastroEE-war/web/WEB-INF/views/rodape.jspf

```
</main>
</body>
</html>
```

CadastroEE-war/web/WEB-INF/web.xml

```xml
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns="https://jakarta.ee/xml/ns/jakartaee" version="6.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="https://jakarta.ee/xml/ns/jakartaee
https://jakarta.ee/xml/ns/jakartaee/web-app_6_0.xsd">
    <display-name>Cadastro de produtos</display-name>
    <servlet>
        <servlet-name>ServletProduto</servlet-name>
        <servlet-class>cadastroee.servlets.ServletProduto</servlet-class>
    </servlet>
    <servlet>
        <servlet-name>ServletProdutoFC</servlet-name>
        <servlet-class>cadastroee.servlets.ServletProdutoFC</servlet-class>
    </servlet>
    <servlet-mapping>
        <servlet-name>ServletProduto</servlet-name>
        <url-pattern>/ServletProduto</url-pattern>
    </servlet-mapping>
    <servlet-mapping>
        <servlet-name>ServletProdutoFC</servlet-name>
        <url-pattern>/ServletProdutoFC</url-pattern>
    </servlet-mapping>
    <welcome-file-list><welcome-file>ServletProdutoFC</welcome-file></welcome-file-list>
    <session-config>
        <session-timeout>30</session-timeout>
        <cookie-config><http-only>true</http-only></cookie-config>
        <tracking-mode>COOKIE</tracking-mode>
    </session-config>
    <error-page>
        <error-code>500</error-code>
        <location>/WEB-INF/views/Erro.jsp</location>
    </error-page>
</web-app>
```

CadastroEE/src/conf/application.xml

```xml
<?xml version="1.0" encoding="UTF-8"?>
<application xmlns="https://jakarta.ee/xml/ns/jakartaee" version="10"
             xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
             xsi:schemaLocation="https://jakarta.ee/xml/ns/jakartaee
https://jakarta.ee/xml/ns/jakartaee/application_10.xsd">
    <display-name>CadastroEE</display-name>
    <module><ejb>CadastroEE-ejb.jar</ejb></module>
    <module>

<web><web-uri>CadastroEE-war.war</web-uri><context-root>/CadastroEE-war</context-root></web>
    </module>
</application>
```

database/loja.sql

```sql
IF DB_ID(N'loja') IS NULL
    CREATE DATABASE loja;
GO
USE loja;
GO
IF OBJECT_ID(N'dbo.Produto', N'U') IS NULL
BEGIN
    CREATE TABLE dbo.Produto (
        idProduto INT IDENTITY(1,1) PRIMARY KEY,
        nome NVARCHAR(100) NOT NULL,
        quantidade INT NOT NULL CHECK (quantidade >= 0),
        precoVenda REAL NOT NULL CHECK (precoVenda >= 0)
    );
END;
GO
IF NOT EXISTS (SELECT 1 FROM dbo.Produto)
BEGIN
    INSERT INTO dbo.Produto (nome, quantidade, precoVenda) VALUES
        (N'Caneta', 50, 2.50),
```

```
        (N'Caderno', 20, 18.90),
        (N'Mochila', 10, 89.90);
END;
GO
```

scripts/configurar-glassfish.ps1

```
param(
    [Parameter(Mandatory = $true)][string]$GlassFishHome,
    [string]$Servidor = 'localhost',
    [int]$Porta = 1433,
    [string]$Usuario = 'loja'
)

$ErrorActionPreference = 'Stop'
if (-not $env:LOJA_PASSWORD) {
    throw 'Defina LOJA_PASSWORD com a senha do usuário do banco antes de executar.'
}
$asadmin = Join-Path $GlassFishHome 'bin/asadmin.bat'
$arquivo = New-TemporaryFile
try {
    $xml = [xml]@'
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE resources PUBLIC "-//GlassFish.org//DTD GlassFish Application Server 3.1 Resource Definitions//EN" "http://glassfish.org/dtds/glassfish-resources_1_5.dtd">
<resources>
    <jdbc-connection-pool name="SQLServerPool" res-type="javax.sql.DataSource"
        datasource-classname="com.microsoft.sqlserver.jdbc.SQLServerDataSource"
        steady-pool-size="2" max-pool-size="10" is-connection-validation-required="true"
        connection-validation-method="table" validation-table-name="dbo.Produto">
        <property name="serverName" value=""/>
        <property name="portNumber" value=""/>
        <property name="databaseName" value="loja"/>
        <property name="user" value=""/>
        <property name="password" value=""/>
        <property name="encrypt" value="true"/>
        <property name="trustServerCertificate" value="true"/>
    </jdbc-connection-pool>
    <jdbc-resource jndi-name="jdbc/loja" pool-name="SQLServerPool"/>
</resources>
'@
    $valores = @{ serverName = $Servidor; portNumber = "$Porta"; user = $Usuario; password = $env:LOJA_PASSWORD }
    foreach ($propriedade in $xml.resources.'jdbc-connection-pool'.property) {
        if ($valores.ContainsKey($propriedade.name)) {
            $propriedade.SetAttribute('value', $valores[$propriedade.name])
        }
    }
    $xml.Save($arquivo.FullName)
    & $asadmin add-resources $arquivo.FullName
    if ($LASTEXITCODE -ne 0) { throw 'Não foi possível criar os recursos JDBC.' }
    & $asadmin ping-connection-pool SQLServerPool
    if ($LASTEXITCODE -ne 0) { throw 'O pool não conseguiu acessar o banco loja.' }
} finally {
    Remove-Item -LiteralPath $arquivo.FullName -Force
}
```

test/cadastroee/model/ProdutoTest.java

```
package cadastroee.model;

public class ProdutoTest {
    private static int verificacoes;

    public static void main(String[] args) {
        Produto produto = new Produto("  Caderno  ", 10, 18.90f);
        conferir("Caderno".equals(produto.getNome()));
        conferir(produto.getIdProduto() == null);
        produto.atualizar("Caneta", 0, 0f);
        conferir(produto.getQuantidade() == 0 && produto.getPrecoVenda() == 0f);
        rejeitar(() -> produto.atualizar("", 1, 1f));
        rejeitar(() -> produto.atualizar(" ", 1, 1f));
```

```
        rejeitar(() -> produto.atualizar(null, 1, 1f));
        rejeitar(() -> produto.atualizar("a".repeat(101), 1, 1f));
        rejeitar(() -> produto.atualizar("Produto", -1, 1f));
        rejeitar(() -> produto.atualizar("Produto", null, 1f));
        rejeitar(() -> produto.atualizar("Produto", 1, -1f));
        rejeitar(() -> produto.atualizar("Produto", 1, null));
        rejeitar(() -> produto.atualizar("Produto", 1, Float.NaN));
        rejeitar(() -> produto.atualizar("Produto", 1, Float.POSITIVE_INFINITY));
        conferir("Caneta".equals(produto.getNome()) && produto.getQuantidade() == 0);
        System.out.println(verificacoes + " verificações do domínio passaram.");
    }

    private static void rejeitar(Runnable operacao) {
        try {
            operacao.run();
        } catch (IllegalArgumentException expected) {
            verificacoes++;
            return;
        }
        throw new AssertionError("A operação deveria ter sido rejeitada.");
    }

    private static void conferir(boolean condicao) {
        if (!condicao) {
            throw new AssertionError("Resultado diferente do esperado.");
        }
        verificacoes++;
    }
}
```

test/integracao.py

```
"""Teste HTTP contra uma instalação local; cria e remove somente seus produtos."""
import html
import http.cookiejar
import re
import sys
import urllib.error
import urllib.parse
import urllib.request
import uuid

base = (sys.argv[1] if len(sys.argv) > 1 else
'http://localhost:8080/CadastroEE-war').rstrip('/')
url = base + '/ServletProdutoFC'
client =
urllib.request.build_opener(urllib.request.HTTPCookieProcessor(http.cookiejar.CookieJar()))
checks = 0
created = []


def request(params, method='GET'):
    encoded = urllib.parse.urlencode(params)
    target = url + '?' + encoded if method == 'GET' else url
    data = encoded.encode() if method == 'POST' else None
    try:
        response = client.open(target, data, timeout=30)
    except urllib.error.HTTPError as error:
        response = error
    return response.status, response.read().decode('utf-8')


def check(condition, message):
    global checks
    if not condition:
        raise AssertionError(message)
    checks += 1
    print('OK - ' + message, flush=True)


def token():
    status, page = request({'acao': 'formIncluir'})
```

```
    if status != 200:
        raise AssertionError('Formulário indisponível: ' + str(status))
    return re.search(r'name="csrf" value="([^"]+)"', page).group(1)


def post(action, **values):
    return request({'acao': action, 'csrf': token(), **values}, 'POST')


try:
    status, page = request({'acao': 'listar'})
    check(status == 200 and 'Produtos cadastrados' in page, 'Listagem com JSP e EJB')
    check('table-striped' in page and 'bootstrap@5.3.8' in page, 'Bootstrap na listagem')
    check('Preço de venda' in page and 'Ações' in page, 'Acentuação UTF-8 na listagem')
    status, page = request({'acao': 'formIncluir'})
    check(status == 200 and 'name="nome"' in page, 'Formulário de inclusão')
    check('Preço de venda' in page, 'Acentuação UTF-8 no formulário')
    name = 'Teste ' + uuid.uuid4().hex[:8] + ' <script>alert(1)</script>'
    first_token = token()
    params = {'acao': 'incluir', 'csrf': first_token, 'nome': name, 'quantidade': '7',
'precoVenda': '12.50'}
    status, page = request(params, 'POST')
    check(status == 200 and 'Produto incluído.' in page, 'Inclusão persistida')
    row = next(row for row in re.findall(r'<tr>(.*?)</tr>', page, re.S) if html.escape(name)
 in row)
    product_id = re.search(r'acao=formAlterar&amp;id=(\d+)', row).group(1)
    created.append(product_id)
    check(name not in page and html.escape(name) in page, 'Nome escapado contra XSS')
    check(request(params, 'POST')[0] == 403, 'Reenvio do mesmo formulário bloqueado')
    status, page = request({'acao': 'formAlterar', 'id': product_id})
    check(status == 200 and html.escape(name) in page, 'Formulário recupera a entidade')
    status, page = post('alterar', id=product_id, nome='Produto atualizado', quantidade='8',
 precoVenda='15,75')
    check(status == 200 and 'Produto atualizado.' in page, 'Alteração persistida e preço com
 vírgula')
    status, page = request({'acao': 'formAlterar', 'id': product_id})
    check('value="8"' in page and 'value="15.75"' in page, 'Nova consulta confirma os
valores gravados')
    status, _ = request({'acao': 'excluir', 'id': product_id})
    check(status == 200 and request({'acao': 'formAlterar', 'id': product_id})[0] == 200,
          'GET de exclusão apenas pede confirmação')
    for field, value in [('nome', ''), ('nome', 'x' * 101), ('quantidade', '-1'),
                         ('quantidade', '1.5'), ('quantidade', '2147483648'),
                         ('precoVenda', '-1'), ('precoVenda', 'NaN'), ('precoVenda',
'Infinity')]:
        values = {'nome': 'Teste inválido', 'quantidade': '1', 'precoVenda': '1'}
        values[field] = value
        status, page = post('incluir', **values)
        check(status == 400 and 'alert-danger' in page, f'Validação: {field}={value[:20]}')
    check(request({'acao': 'incluir', 'nome': 'Sem token'}, 'POST')[0] == 403, 'POST sem
token rejeitado')
    check(request({'acao': 'incluir'})[0] == 405, 'Inclusão por GET rejeitada')
    check(request({'acao': 'alterar'})[0] == 405, 'Alteração por GET rejeitada')
    check(request({'acao': 'desconhecida'})[0] == 400, 'Ação desconhecida rejeitada')
    check(request({'acao': 'formAlterar', 'id': 'abc'})[0] == 400, 'ID inválido rejeitado')
    check(request({'acao': 'formAlterar', 'id': '2147483647'})[0] == 404, 'Produto
inexistente retorna 404')
    check(client.open(base + '/ServletProduto', timeout=30).status == 200, 'Servlet de teste
 funciona')
    status, page = post('excluir', id=product_id)
    check(status == 200 and 'Produto excluído.' in page, 'Exclusão persistida')
    check(request({'acao': 'formAlterar', 'id': product_id})[0] == 404, 'Consulta confirma a
 exclusão')
    created.remove(product_id)
    print(f'\n{checks} verificações HTTP passaram.')
finally:
    for product_id in created:
        post('excluir', id=product_id)
```

nbproject/project.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://www.netbeans.org/ns/project/1">
    <type>org.netbeans.modules.ant.freeform</type>
    <configuration>
        <general-data xmlns="http://www.netbeans.org/ns/freeform-project/2">
            <name>CadastroEE</name>
            <properties/>
            <folders>
                <source-folder><label>CadastroEE-ejb</label><type>java</type><location>CadastroEE-ejb/src/java</location><encoding>UTF-8</encoding></source-folder>
                <source-folder><label>CadastroEE-war</label><type>java</type><location>CadastroEE-war/src/java</location><encoding>UTF-8</encoding></source-folder>
                <source-folder><label>Testes</label><type>java</type><location>test</location><encoding>UTF-8</encoding></source-folder>
            </folders>
            <ide-actions>
                <action name="build"><target>dist</target></action>
                <action name="clean"><target>clean</target></action>
                <action name="rebuild"><target>clean</target><target>dist</target></action>
                <action name="test"><target>test</target></action>
                <action name="run"><target>deploy</target></action>
            </ide-actions>
            <view>
                <items>
                    <source-folder style="packages"><label>CadastroEE-ejb</label><location>CadastroEE-ejb/src/java</location></source-folder>
                    <source-folder style="packages"><label>CadastroEE-war</label><location>CadastroEE-war/src/java</location></source-folder>
                    <source-folder style="tree"><label>Páginas JSP</label><location>CadastroEE-war/web</location></source-folder>
                    <source-folder style="packages"><label>Testes</label><location>test</location></source-folder>
                    <source-file><location>build.xml</location></source-file>
                </items>
                <context-menu><ide-action name="build"/><ide-action name="rebuild"/><ide-action name="clean"/><ide-action name="test"/><ide-action name="run"/></context-menu>
            </view>
        </general-data>
        <java-data xmlns="http://www.netbeans.org/ns/freeform-project-java/4">
            <compilation-unit>
                <package-root>CadastroEE-ejb/src/java</package-root>
                <classpath mode="compile">lib/jakarta.jakartaee-api-10.0.0.jar</classpath>
                <built-to>build/ejb</built-to><source-level>17</source-level>
            </compilation-unit>
            <compilation-unit>
                <package-root>CadastroEE-war/src/java</package-root>
                <classpath mode="compile">lib/jakarta.jakartaee-api-10.0.0.jar:build/ejb</classpath>
                <built-to>build/war</built-to><source-level>17</source-level>
            </compilation-unit>
        </java-data>
    </configuration>
</project>
```

build.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project name="CadastroEE" default="dist" basedir=".">
    <property environment="env"/>
    <property name="api.jar" location="lib/jakarta.jakartaee-api-10.0.0.jar"/>
    <property name="asadmin" location="${env.GLASSFISH_HOME}/bin/asadmin"/>

    <target name="dependencies">
        <mkdir dir="lib"/>
        <get src="https://repo.maven.apache.org/maven2/jakarta/platform/jakarta.jakartaee-api/10.0.0/jakarta.jakartaee-api-10.0.0.jar"
             dest="${api.jar}" skipexisting="true"/>
    </target>

```

```
    <target name="compile" depends="dependencies">
        <mkdir dir="build/ejb"/>
        <mkdir dir="build/war"/>
        <javac srcdir="CadastroEE-ejb/src/java" destdir="build/ejb" release="17"
               encoding="UTF-8" includeantruntime="false" classpath="${api.jar}"/>
        <javac srcdir="CadastroEE-war/src/java" destdir="build/war" release="17"
               encoding="UTF-8" includeantruntime="false" classpath="${api.jar};build/ejb"/>
    </target>

    <target name="test" depends="compile">
        <mkdir dir="build/test"/>
        <javac srcdir="test" destdir="build/test" release="17" encoding="UTF-8"
               includeantruntime="false" classpath="${api.jar};build/ejb"/>
        <java classname="cadastroee.model.ProdutoTest" fork="true" failonerror="true"
              classpath="${api.jar};build/ejb;build/test"/>
    </target>

    <target name="dist" depends="test">
        <mkdir dir="dist"/>
        <jar destfile="dist/CadastroEE-ejb.jar" basedir="build/ejb">
            <metainf dir="CadastroEE-ejb/src/conf"/>
        </jar>
        <war destfile="dist/CadastroEE-war.war" webxml="CadastroEE-war/web/WEB-INF/web.xml">
            <fileset dir="CadastroEE-war/web" excludes="WEB-INF/web.xml"/>
            <classes dir="build/war"/>
        </war>
        <ear destfile="dist/CadastroEE.ear" appxml="CadastroEE/src/conf/application.xml">
            <fileset dir="dist" includes="CadastroEE-ejb.jar,CadastroEE-war.war"/>
        </ear>
    </target>

    <target name="deploy" depends="dist" description="Implanta no GlassFish iniciado;
configure -Dasadmin=caminho completo.">
        <exec executable="${asadmin}" failonerror="true">
            <arg value="deploy"/><arg value="--force=true"/><arg
file="dist/CadastroEE.ear"/>
        </exec>
    </target>

    <target name="clean">
        <delete dir="build"/><delete dir="dist"/>
    </target>
</project>
```